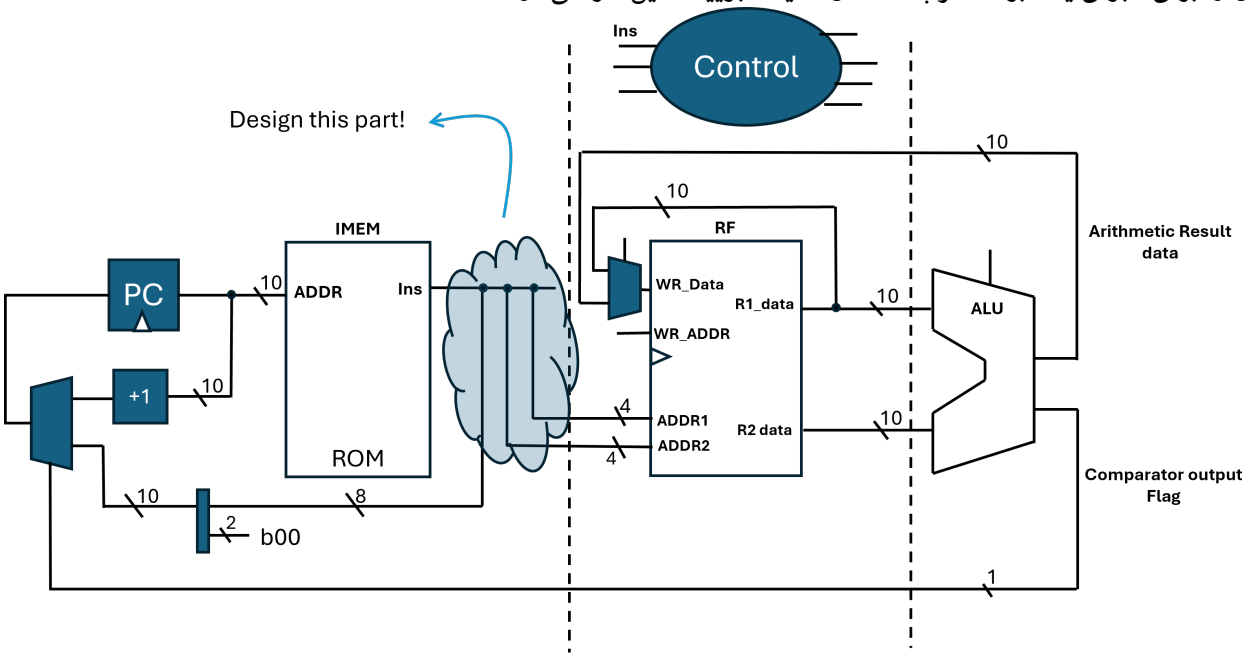
پروژه طراحی پردازنده ساده تک سیکلی

دانشکده مهندسی برق دانشگاه شریف
دکتر سعید باقری شورکی

پروژه میانی درس
مهلت تحویل : ۵ خرداد

در آزمایش‌های پیشین با اصول و الفبای اولیه طراحی یک پردازنده آشنا شدید. در این پروژه قصد داریم یک پردازنده کوچک طراحی کنیم. یک پردازنده ساده با مجموعه دستورالعمل‌هایی (ISA) [1] که شامل دستورات محاسباتی، مقایسه و کنترلی است، تعریف و بلوک‌های ساختاری آن ساخته می‌شود. طبق بخش طراحی پردازنده، عملکرد آن را شبیه‌سازی کنید و توانایی آن را برای اجرای یک برنامه کوچک نشان دهید. جزییات این طراحی در ادامه آمده است.



شکل ۱: بلوک دیاگرام ساختار پردازنده

توجه فرمایید بلوک دیاگرام به هیچ عنوان کامل نمی‌باشد و در ورودی رجیسترفایل نیاز به مالتی پلکسرهایی خواهد بود. در این ساختار سعی خواهد شد دنباله‌ای از دستورات به صورت خودکار اجرا شده تا در نهایت به یک هدف مشخص دست یابیم. این دنباله از دستورات که با توجه به فرمت ارایه شده نوشته می‌شود در یک حافظه ریخته می‌شود. یک شمارنده این دنباله را اجرا می‌کند و با اتمام اجرای آن دستور به سراغ دستور بعدی می‌رود. در برخی اوقات می‌تواند از دستوراتی پرش کرده یا به عقب بازگردد. پس از خوانش دستورات، آن‌ها دیکود می‌شوند و سیگنال‌های کنترلی را بوجود می‌آورند تا عملیات‌ها در صورت لزوم ذخیره شده یا به هدفی که می‌خواهیم برسند.

[1] Instruction Set Architecture

۱ طراحی پردازنده

در این پروژه، هدف طراحی یک سیستم کنترل هوشمند برای مدیریت توان یک خودروی برقی (EV)است. فرض کنید این خودرو در حالت حرکت عادی، در هر واحد زمانی ۴ واحد از شارژ باتری خود را مصرف می‌کند. سطح شارژ اولیه باتری ۸۰ واحد است.

برای جلوگیری از تخلیه کامل، یک سیستم «ژنراتور پشتیبان» یا «حالت بازیابی انرژی» تعبیه شده است. منطق کنترل به شرح زیر است:

- خودرو در ابتدا در حال تخلیه باتری است.
- زمانی که سطح شارژ باتری به کمتر از ۲۰ واحد رسید، سیستم شارژ مجدد فعال می‌شود.
- در حالت شارژ، در هر واحد زمانی ۶ واحد به شارژ باتری اضافه می‌شود (مصرف متوقف شده یا نرخ شارژ بر مصرف غلبه می‌کند).
- عملیات شارژ تا زمانی که سطح باتری به ۹۰ واحد برسد ادامه یافته و پس از آن سیستم شارژ خاموش شده و خودرو مجدداً به حالت مصرف (تخلیه) باز می‌گردد.

از شما می‌خواهیم ساختار پردازنده را به صورت **۱۰ بیتی** طراحی کرده و برنامه اسمبلی بنویسید که این چرخه مدیریت شارژ را پیاده‌سازی کند. یک خروجی (۷ سگمنت) باید وضعیت «در حال شارژ» را در طول شبیه‌سازی نمایش دهد. صرفا پیاده‌سازی یک چرخه کامل (تخلیه تا ۲۰ و شارژ تا ۹۰) کافی است.

۲ اجزای تشکیل دهنده این پردازنده

۱.۲ رجیسترها

این پردازنده دارای رجیسترهای زیر است:

۱- **شمارنده برنامه یا PC** [۲]آدرس دستورهای ذخیره شده در حافظه را در خود ذخیره می‌کند و به آن خانه حافظه اشاره می‌کند. با اجرای هر دستور یک واحد به مقدار این شمارنده اضافه می‌شود. مقدار این رجیستر می‌تواند ریست شود و یا مقدار جدیدی بگیرد، به عبارتی با ارضا شدن یک شرطی پرش صورت بگیرد. معمولا در آغاز به خانه اول حافظه دستورات اشاره دارد. این شمارنده PC مانند یک sequencer عمل می کند. در نرم‌افزار Digital ، برای PC می‌توانید از کامپوننت counter استفاده کنید.

۲- **رجیسترفایل**: برای ساماندهی رجیسترها در طراحی پردازنده‌ها از رجیسترفایل استفاده می‌شود. رجیسترفایل مورد نظر دارای ۱۶ رجیستر داخلی است و عرض داده‌ها ۱۰ بیت هستند. داده ذخیره شده در هر خانه با یک آدرس ۴ بیتی قابل دسترس است. از این رجیسترها برای نگهداری مقادیر مورد نیاز برای انجام محاسبات در ALU و مقادیر میانی و موقت حاصل از نتایج محاسبات استفاده می‌شود. این رجیسترفایل می‌تواند دو پورت خواندن و تنها یک پورت نوشتن داشته باشد.

[2]Program Counter (PC)

نحوه نوشتن و خواندن با سیگنال‌های کنترلی مشخص می‌شوند. در نرم‌افزار Digital در قسمت کامپوننت، حافظه، رم گزینه RegisterFile را انتخاب نمایید. به جای RegisterFile می‌توانید مانند آزمایش ۲ از چندین رجیستر جداگانه استفاده نمایید (توصیه نمی‌شود).

۳- **فلگ‌ها** مانند بزرگ‌تری، کوچکتری و برابری برای تعیین نتیجه خروجی ALU که در ادامه دستورات مورد استفاده قرار می‌گیرد. با توجه به نوع مسئله طراحی صرفا می‌توان حالت بزرگتری را پیاده کرد.

۲.۲ حافظه EEPROM

این پردازنده همچنین دارای حافظه‌ای است که برای نگهداری دنباله‌ی دستورات که به Instruction Memory یا به اختصار IMEM به کار گرفته می‌شود. دنباله‌ی دستورات معمولا کدهای اسمبلی هستند که از کدهای زبان سطح بالا سنتز شده‌اند. این دستورات اسمبلی را می‌توان به صورت باینری در نرم‌افزار یا به صورت فایل hex پیش از اجرای شبیه‌سازی در ROM مقداردهی کرد. می‌توان از کامپوننت ROM موجود در نرم‌افزار Digital استفاده کرد.

برای تبدیل فایلی که دنباله باینری دستورات در آن ذخیره شده است به فایل hex (فایل مورد نیاز برای مقداردهی حافظه) می‌توانید از کدی که قرار داده شده است مانند آموزش ویدیویی استفاده کنید یا به صورت گرافیکی مقادیر باینری را در تنظیمات ROM وارد کنید.

برای استفاده از این کد صرفا نیاز است زبان پایتون در سیستم شما نصب باشد. یا بدون هیچ پکیج یا نرم‌افزاری کافی است کدها را در گوگل کولب کپی کرده و ران کنید. این کد صرفا برای تسهیل مقداردهی اولیه ROM است. دو ویدیوی آموزشی همراه دستورکار در صفحه درس قرار داده شده است.

۳.۲ فرمت دستورات

برای سادگی و قابلیت پیاده‌سازی این پردازنده در نرم‌افزار Digital، عرض دستورات را ۱۰ بیتی در نظر بگیرید که در آن ۲ بیت اول نشان دهنده نوع دستور است. بیت‌های بعدی بسته به نوع دستور متفاوت هستند. برای مثال در دستورات محاسباتی ۴ بیت بعدی نوع محاسبه و سه بیت آخر آدرس رجیستر برای ذخیره نتایج است. با توجه به اینکه ۲ بیت برای انواع دستورات در نظر گرفته شده است، این چهار نوع دستور در ادامه توضیح داده شده است. به دلیل عرض کم فرمت دستورات از دستورهایی برای بارگذاری مقدار فوری در رجیستر استفاده می‌کنیم سپس با استفاده از سایر دستورات عملیات‌های لازم را بر روی آن انجام می‌دهیم. مجموعه دستورات می‌تواند شامل دستورات زیر باشد:

Instruction	Definition	Expalnation
Addition	ADD Rd	$R_d = R_0 + R_1$
Subtract	SUB Rd	$R_d = R_0 - R_1$
Immediate	IMM Val	$R_5 = Val$
CND	CND Val	Compare Rd with Rs, set flag, then put $Val -> PC$
Move	MOV Rs, Rd	Move Rs to Rd

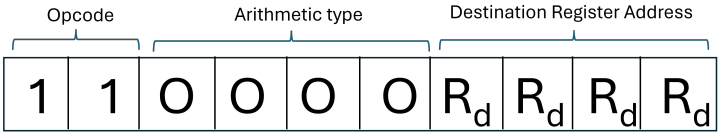
در دستورات IMM و CND دو بیت اول نوع دستور را مشخص می‌کند. ۸ بیت باقی‌مانده مقدار فوری و آدرس خانه دستور را مشخص می‌کنند. از آنجایی که این دوعدد باید ۱۰ بیتی می‌بودند کافی است آن‌ها را از ۸ بیت به ۱۰ بیت extend کنیم.

پس از extend کردن می‌توان آن‌ها را در رجیسترفایل و یا در PC قرار داد. دو بیت افزوده شده را 00 در نظر بگیرید.

۴.۲ واحد پردازش یا ALU

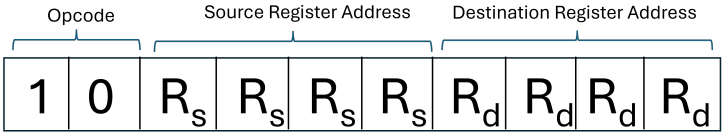
واحد ALU واحدی است که وظیفه انجام محاسبات را دارد. در دو آزمایش قبلی با این عملیات‌ها آشنا شده‌اید. توجه فرمایید مقادیر از رجیسترفایل خوانده شده و محاسبات روی آن‌ها انجام می‌شود و سپس نتیجه در خانه مشخصی از رجیسترفایل ذخیره می‌شود. این محاسبات می‌تواند از نوع جمع، تفریق یا مقایسه باشد.

مثالی از این دستورات را می‌توان در شکل ۲ پیدا کرد که در آن دستور ADD توضیح داده شده است. با اختصاص سه بیت Arithmetic type در کل می‌توان ۱۶ عملیات برای ALU تعریف کرد. نحوه اختصاص این ۴ بیت به عملیات ADD برابر 0000 و 0001 برای SUB است.

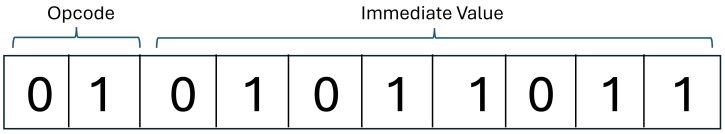


شکل ۲: نحوه تعریف دستورهای محاسباتی مانند ADD

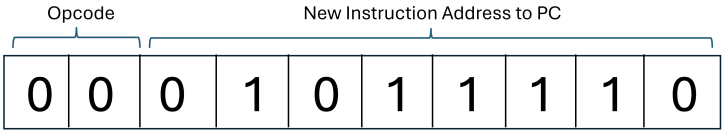
همچنین سایر دستورات به صورت زیر هستند:



شکل ۳: نحوه تعریف دستور MOV



شکل ۴: نحوه تعریف دستور IMM



شکل ۵: نحوه تعریف دستور CND

با بیشتر دستورات در آزمایش‌های قبلی آشنا شده‌اید. علاوه بر دستورات پیشین، در این آزمایش از دستور پرشی [۳] CDN استفاده خواهیم کرد. دستورات پرشی برای پیاده‌سازی حلقه‌ها در زبان‌های برنامه‌نویسی استفاده می‌شوند. می‌توانید دستور

[3] Jump Instruction

CND را صرفا برای بزرگتر بودن مقدار یک رجیستر از رجیستر دیگر را لحاظ کنید و در صورت تحقق یا عدم تحقق آن پرش صورت بگیرد. از این رو لازم است از قبل مقادیر مطلوب با سایر دستورات قبلا در خانه‌های درست رجیستر قرار داده شده باشند. هنگام اجرای دستور CND مقایسه در ALU به صورت ضمنی صورت می‌پذیرد و در صورت تحقق شرط مقدار دستور CND در PC قرار می‌گیرد.

برای راهنمایی در زیر، دنباله دستوری برای یک برنامه ساده نوشته شده است:

در این کد مقادیر ۱۰ و ۱۵ در رجیستر R_5 لود شده و به ترتیب در رجیسترهای R_0 و R_1 ذخیره می‌شوند و با هم جمع شده و مقدار خروجی در رجیستر R_2 و R_3 ذخیره می‌شود. سپس عدد ۲۰ در رجیستر R_5 لود می‌شود. سپس با چک کردن یک فلگ توسط دستور CND عدد ۵ در PC قرار می‌گیرد.

نکته: برای دستورات محاسباتی مانند ADD و SUB، دو رجیستر ثابت R_0 و R_1 برای ورودی عملگر و برای دستور IMM یک رجیستر ثابت مانند R_5 در نظر بگیرید. **چگونگی تعیین آدرس این رجیستر ثابت به این صورت است که مقدار یکان عدد دانشجویی خود را با ۲ جمع کنید مثلا اگر رقم یکان شماره دانشجویی شما برابر ۱ باشد رجیستر R_3 را در نظر بگیرید. عدم رعایت این نکته باعث می شود نمره‌ای به شما تعلق نگیرد!** مقادیر مختلف درون رجیسترهای رجیسترفایل را می‌توان با دستور MOV با هم جابه‌جا کرد.

به دلیل اینکه همواره رجیسترهای R_0 و R_1 به ALU ریخته می‌شوند و نتیجه به رجیسترفایل برمی‌گردد نیاز است برای بهره‌مندی از دستور IMM (که مقداری را در رجیسترفایل می‌نویسد) در پورت نوشتن داده و آدرس رجیسترفایل از مالتی‌پلکسر استفاده کنید.

```
ROM |  Your Assembly |                Explanation                  | Assembly
Addr|  Code          |                                             | To Binary
 ---|----------------|---------------------------------------------|--------------
 0  |  IMM 10        | Load threshold 10 into R5                   | # 0100001010
 1  |  MOV R0, R5    | Load R5 into R0                             | # 1001010000
 2  |  IMM 15        | Load threshold 15 into R5                   | # 0100001111
 3  |  MOV R1, R5    | Load R5 into R1                             | # 1001010001
 4  |  ADD R2        | R2 = R0 + R1                                | # 1100010010
 5  |  ADD R3        | R3 = R0 + R1                                | # 1100010011
 6  |  IMM 20        | Load threshold 20 into R5                   | # 1000010100
 7  |  CND  5        | Check flag for Compared result with threshold | # 0000000101
    |  HLT           | Halt execution                              |
```

با طراحی و انتخاب دستورات مشابه شکل‌های ۲ تا ۵ برای کد بالا، دستورات اسمبلی به اعداد باینری تبدیل می‌شوند که به ترتیب باید در خانه‌های حافظه ROM ذخیره شوند تا در حین کار PC به آن خانه‌ها اشاره داشته باشد.

راهنمایی: برای این که بتوانید انجام پروژه را قدم به قدم پیش ببرید، ابتدا فلوچارت برنامه را رسم نمایید. سپس کدهای

اسمبلی آن را بنویسید. پس از آن با توجه به دستوراتی که در شکل‌های ۲ تا ۵ آمده است دستورات را به باینری تبدیل کنید. با توجه به فرمت دستورات، ساختار و سیگنال‌های کنترلی را رسم نمایید. سپس کدهای باینری را به صورت دستی یا با استفاده از کدی که قرار داده شده است به صورت فایل با فرمت hex. وارد حافظه ROM نمایید. برای چک کردن بخش‌های مختلف برنامه می‌توانید از تست‌های مختلف استفاده کنید.

۳ تحویل پروژه

انجام پروژه به صورت فردی می‌باشد. برای تحویل پروژه نیازی به نوشتن گزارش نیست. تنها نیاز است ویدیویی چند دقیقه‌ای از صفحه نمایشتان که در آن عملکرد مدار را توضیح می‌دهید در CW به همراه فایل‌های پروژه ارسال نمایید. برای این کار می‌توانید از نرم‌افزارهای مختلف مانند ZDsoft یا OBS استفاده کنید. همچنین فایل حاوی کدهای اسمبلی و باینری که در حافظه قرار می‌دهید را همراه فایل پروژه ارسال نمایید.

از آنجایی که آزادی عمل در پیاده‌سازی این پروژه وجود دارد پیاده‌سازی‌ها شبیه هم نخواهند بود و شباهت بیش از اندازه پیاده‌سازی‌ها مورد پذیرش نیست.